

Multiple Linear Regression – Multi-Channel Marketing Analysis

Objective: Build a robust Multiple Linear Regression model to predict `Sales` from TV spend level, Radio spend, Social Media spend, and Influencer type. We will:

- Perform exploratory data analysis (EDA)
- Check for multicollinearity via correlation matrix and VIF
- Fit an OLS regression model with `statsmodels`
- Evaluate performance (Adjusted R^2 , p-values)
- Validate regression assumptions (Linearity, Normality, Homoscedasticity)
- Deliver business-ready coefficient interpretations and budget recommendations

Dataset: `marketing_sales_data.csv` — 572 observations, 5 columns

`TV` (Low/Medium/High), `Radio` (continuous, K), `'SocialMedia'` (*continuous*, K), `Influencer` (Macro/Mega/Micro/Nano), `Sales` (continuous, \$K)

1. Environment Setup & Imports

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor
from scipy import stats

# Display settings
pd.set_option('display.float_format', '{:.4f}'.format)
plt.rcParams['figure.figsize'] = (10, 6)
sns.set_style('whitegrid')
sns.set_palette('muted')

print('All libraries imported successfully!')
```

All libraries imported successfully!

2. Load & Explore the Dataset

```
df = pd.read_csv('marketing_sales_data.csv')

print('Shape:', df.shape)
print()
df.head()
```

Shape: (572, 5)

	TV	Radio	Social Media	Influencer	Sales
0	Low	3.5181	2.2938	Micro	55.2613
1	Low	7.7569	2.5723	Mega	67.5749
2	High	20.3490	1.2272	Micro	272.2501
3	Medium	20.1085	2.7284	Mega	195.1022
4	High	31.6532	7.7770	Nano	273.9604

```
# Data types and missing values
print('Data Types:\n', df.dtypes)
print('\nMissing Values:\n', df.isnull().sum())
```

```
Data Types:
TV                str
Radio             float64
Social Media      float64
Influencer        str
```

```
Sales          float64
dtype: object

Missing Values:
TV              0
Radio           0
Social Media    0
Influencer      0
Sales           0
dtype: int64
```

```
# Statistical summary of numeric columns
df.describe()
```

	Radio	Social Media	Sales
count	572.0000	572.0000	572.0000
mean	17.5206	3.3338	189.2969
std	9.2909	2.2384	89.8716
min	0.1091	0.0000	33.5098
25%	10.6996	1.5855	118.7187
50%	17.1495	3.1501	184.0054
75%	24.6064	4.7304	264.5001
max	42.2716	11.4036	357.7882

```
# Categorical column value counts
print('TV levels:\n', df['TV'].value_counts())
print('\nInfluencer types:\n', df['Influencer'].value_counts())
```

```
TV levels:
TV
Medium    205
Low        201
High       166
Name: count, dtype: int64
```

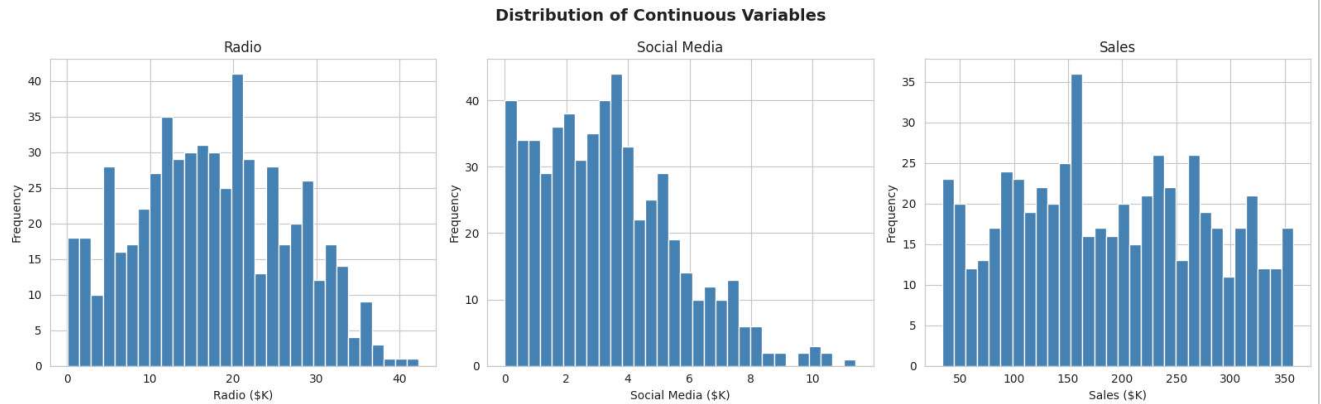
```
Influencer types:
Influencer
Nano       157
Micro      151
Mega       137
Macro      127
Name: count, dtype: int64
```

2.1 EDA Visualisations

```
fig, axes = plt.subplots(1, 3, figsize=(16, 5))
fig.suptitle('Distribution of Continuous Variables', fontsize=14, fontweight='bold')

for ax, col in zip(axes, ['Radio', 'Social Media', 'Sales']):
    ax.hist(df[col], bins=30, edgecolor='white', color='steelblue')
    ax.set_title(col)
    ax.set_xlabel(f'{col} ($K)')
    ax.set_ylabel('Frequency')

plt.tight_layout()
plt.show()
```



```
# Sales vs categorical predictors
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
fig.suptitle('Sales by Categorical Channels', fontsize=14, fontweight='bold')

tv_order = ['Low', 'Medium', 'High']
sns.boxplot(data=df, x='TV', y='Sales', order=tv_order, ax=axes[0], palette='Blues')
axes[0].set_title('Sales by TV Spend Level')

sns.boxplot(data=df, x='Influencer', y='Sales', ax=axes[1], palette='Greens')
axes[1].set_title('Sales by Influencer Type')

plt.tight_layout()
plt.show()
```

/tmp/ipykernel_631/3821301149.py:6: FutureWarning:

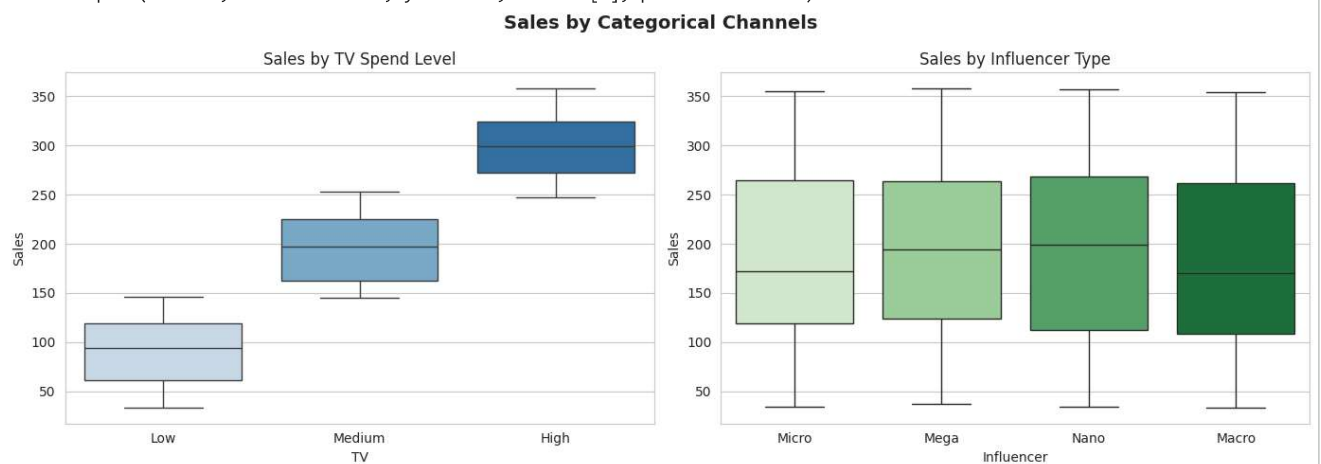
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set

```
sns.boxplot(data=df, x='TV', y='Sales', order=tv_order, ax=axes[0], palette='Blues')
```

/tmp/ipykernel_631/3821301149.py:9: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set

```
sns.boxplot(data=df, x='Influencer', y='Sales', ax=axes[1], palette='Greens')
```

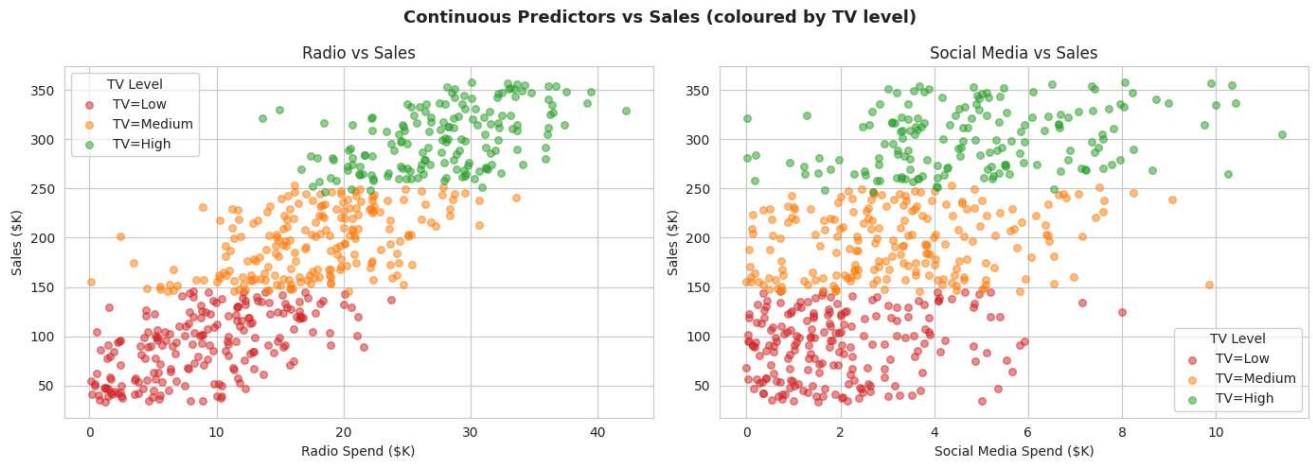


```
# Scatter plots: Radio & Social Media vs Sales, coloured by TV level
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
fig.suptitle('Continuous Predictors vs Sales (coloured by TV level)', fontsize=13, fontweight='bold')

palette = {'Low': '#d62728', 'Medium': '#ff7f0e', 'High': '#2ca02c'}
for ax, col in zip(axes, ['Radio', 'Social Media']):
    for level in tv_order:
        subset = df[df['TV'] == level]
```

```
ax.scatter(subset[col], subset['Sales'], label=f'TV={level}',
           alpha=0.5, color=palette[level], s=30)
ax.set_xlabel(f'{col} Spend ($K)')
ax.set_ylabel('Sales ($K)')
ax.set_title(f'{col} vs Sales')
ax.legend(title='TV Level')

plt.tight_layout()
plt.show()
```



EDA observations:

- Sales varies dramatically with TV level — **High** TV clearly drives the most sales, confirming it's a key predictor.
- Radio shows a positive association with Sales, especially at High TV levels.
- Social Media's relationship with Sales appears weaker — this may not be a significant predictor.
- Influencer type shows minimal variation in Sales median — likely a weak predictor.

3. Feature Engineering & Encoding

TV is **ordinal** (Low < Medium < High), so we encode it as 0/1/2.

Influencer is **nominal** — we apply one-hot encoding (dropping one level to avoid the dummy variable trap).

```
# Ordinal encoding for TV
tv_map = {'Low': 0, 'Medium': 1, 'High': 2}
df['TV_encoded'] = df['TV'].map(tv_map)

# One-hot encoding for Influencer (Macro is the reference/dropped category)
df_model = pd.get_dummies(df, columns=['Influencer'], drop_first=True)
df_model.drop(columns=['TV'], inplace=True) # remove original string column

# Convert boolean dummy columns to int
bool_cols = df_model.select_dtypes(include='bool').columns
df_model[bool_cols] = df_model[bool_cols].astype(int)

print('Processed columns:', df_model.columns.tolist())
df_model.head()
```

Processed columns: ['Radio', 'Social Media', 'Sales', 'TV_encoded', 'Influencer_Mega', 'Influencer_Micro', 'Influencer_Nano']

	Radio	Social Media	Sales	TV_encoded	Influencer_Mega	Influencer_Micro	Influencer_Nano
0	3.5181	2.2938	55.2613	0	0	1	0
1	7.7569	2.5723	67.5749	0	1	0	0
2	20.3490	1.2272	272.2501	2	0	1	0
3	20.1085	2.7284	195.1022	1	1	0	0
4	31.6532	7.7770	273.9604	2	0	0	1

4. Multicollinearity Check

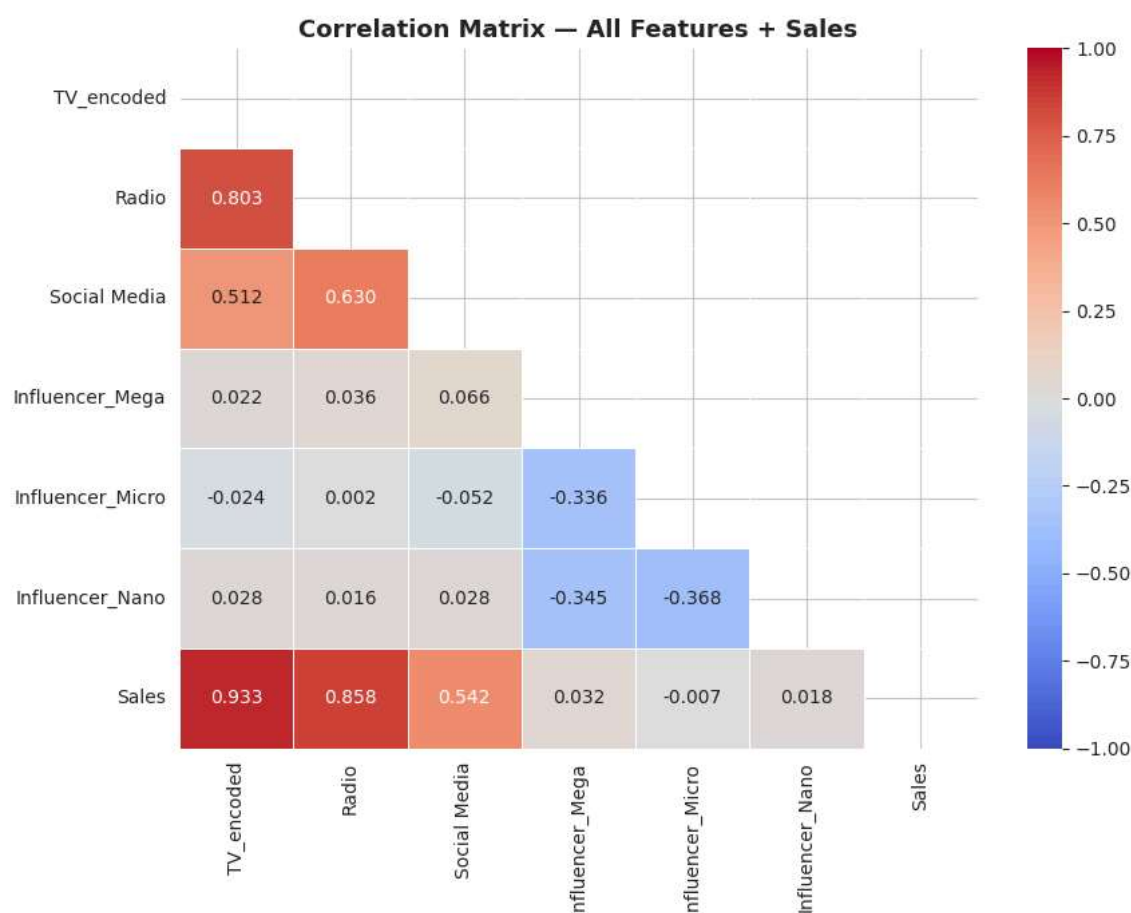
Multicollinearity occurs when predictors are highly correlated with each other. It inflates standard errors and makes coefficient estimates unstable. We check using:

- **Correlation Matrix** — visual inspection
- **Variance Inflation Factor (VIF)** — values > 5 indicate problematic multicollinearity; > 10 is severe

```
feature_cols = ['TV_encoded', 'Radio', 'Social Media',
                'Influencer_Mega', 'Influencer_Micro', 'Influencer_Nano']

corr_matrix = df_model[feature_cols + ['Sales']].corr()

plt.figure(figsize=(9, 7))
mask = np.triu(np.ones_like(corr_matrix, dtype=bool))
sns.heatmap(corr_matrix, annot=True, fmt='.3f', cmap='coolwarm',
            mask=mask, vmin=-1, vmax=1, linewidths=0.5)
plt.title('Correlation Matrix — All Features + Sales', fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()
```



```
# Variance Inflation Factor (VIF)
X_vif = df_model[feature_cols].copy()
X_vif_const = sm.add_constant(X_vif)

vif_data = pd.DataFrame({
    'Feature': X_vif_const.columns,
    'VIF': [variance_inflation_factor(X_vif_const.values, i)
           for i in range(X_vif_const.shape[1])]
})

vif_data = vif_data[vif_data['Feature'] != 'const'].reset_index(drop=True)
print('VIF Scores:')
print(vif_data.to_string(index=False))

# Highlight any VIF > 5
high_vif = vif_data[vif_data['VIF'] > 5]
```

```
if len(high_vif) == 0:
    print('\n✅ No features with VIF > 5. Multicollinearity is not a concern.')
else:
    print(f'\n⚠️ High VIF features (>5): {high_vif["Feature"].tolist()}')
```

VIF Scores:

Feature	VIF
TV_encoded	2.8275
Radio	3.4676
Social Media	1.6686
Influencer_Mega	1.5895
Influencer_Micro	1.6180
Influencer_Nano	1.6264

✅ No features with VIF > 5. Multicollinearity is not a concern.

5. Full Model – Multiple Linear Regression (All Predictors)

We begin with all predictors and use Adjusted R^2 and p-values to decide what to keep.

```
X_full = sm.add_constant(df_model[feature_cols])
y = df_model['Sales']

model_full = sm.OLS(y, X_full).fit()
print(model_full.summary())
```

```
=====
                        OLS Regression Results
=====
```

Dep. Variable:	Sales	R-squared:	0.904
Model:	OLS	Adj. R-squared:	0.903
Method:	Least Squares	F-statistic:	887.9
Date:	Thu, 18 Jun 2026	Prob (F-statistic):	7.41e-284
Time:	20:25:38	Log-Likelihood:	-2713.7
No. Observations:	572	AIC:	5441.
Df Residuals:	565	BIC:	5472.
Df Model:	6		
Covariance Type:	nonrobust		

```
=====
```

	coef	std err	t	P> t	[0.025	0.975]
const	63.5316	3.332	19.065	0.000	56.986	70.077
TV_encoded	77.4451	2.463	31.445	0.000	72.608	82.283
Radio	2.9640	0.235	12.631	0.000	2.503	3.425
Social Media	-0.1469	0.676	-0.217	0.828	-1.474	1.180
Influencer_Mega	2.6192	3.456	0.758	0.449	-4.168	9.407
Influencer_Micro	2.9769	3.376	0.882	0.378	-3.653	9.607
Influencer_Nano	0.7448	3.343	0.223	0.824	-5.822	7.311

```
=====
```

Omnibus:	57.984	Durbin-Watson:	1.876
Prob(Omnibus):	0.000	Jarque-Bera (JB):	17.744
Skew:	0.063	Prob(JB):	0.000140
Kurtosis:	2.146	Cond. No.	93.6

```
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Interpreting the Full Model Summary

Key numbers to read from the table above:

- **Adj. R^2** — the proportion of variance in Sales explained by the model, penalised for the number of predictors. Closer to 1 is better.
- **P>|t|** — p-value for each coefficient. Values < **0.05** indicate statistical significance (we can confidently say that predictor affects Sales).
- **coef** — the expected change in Sales for a one-unit increase in that predictor, holding all others constant.

```
# Extract and display p-values clearly
pval_df = pd.DataFrame({
    'Coefficient': model_full.params,
    'p-value': model_full.pvalues,
    'Significant (p<0.05)': model_full.pvalues < 0.05
}).drop(index='const')

print('Predictor Significance Summary:')
print(pval_df.to_string())
```

Predictor Significance Summary:

	Coefficient	p-value	Significant (p<0.05)
TV_encoded	77.4451	0.0000	True
Radio	2.9640	0.0000	True
Social Media	-0.1469	0.8279	False
Influencer_Mega	2.6192	0.4488	False
Influencer_Micro	2.9769	0.3782	False
Influencer_Nano	0.7448	0.8238	False

6. Refined Model – Keeping Only Significant Predictors

Based on the p-values above, we remove predictors with $p \geq 0.05$ (not statistically significant). This typically yields a higher Adjusted R^2 and cleaner interpretations.

```
# Select features significant at p < 0.05 from the full model
significant_features = pval_df[pval_df['Significant (p<0.05)']].index.tolist()
print('Significant features retained:', significant_features)

X_refined = sm.add_constant(df_model[significant_features])
model_refined = sm.OLS(y, X_refined).fit()
print()
print(model_refined.summary())
```

Significant features retained: ['TV_encoded', 'Radio']

```
=====
                        OLS Regression Results
=====
Dep. Variable:          Sales    R-squared:                0.904
Model:                  OLS    Adj. R-squared:              0.904
Method:                 Least Squares    F-statistic:         2676.
Date:                   Thu, 18 Jun 2026    Prob (F-statistic):    3.70e-290
Time:                   20:25:38    Log-Likelihood:       -2714.3
No. Observations:       572    AIC:                   5435.
Df Residuals:           569    BIC:                   5448.
Df Model:               2
Covariance Type:        nonrobust
=====
                        coef    std err          t      P>|t|      [0.025     0.975]
-----
const                64.9254      2.580     25.162     0.000     59.857     69.994
TV_encoded           77.3156      2.454     31.512     0.000     72.497     82.135
Radio                2.9558      0.211     14.002     0.000      2.541      3.370
=====
Omnibus:                 60.364    Durbin-Watson:           1.873
Prob(Omnibus):           0.000    Jarque-Bera (JB):         17.980
Skew:                    0.050    Prob(JB):                 0.000125
Kurtosis:                 2.137    Cond. No.                  48.3
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
# Compare full vs refined model
comparison = pd.DataFrame({
    'Full Model': {
        'R²': model_full.rsquared,
        'Adjusted R²': model_full.rsquared_adj,
        'AIC': model_full.aic,
        'BIC': model_full.bic,
        'Predictors': len(feature_cols)
    },
    'Refined Model': {
        'R²': model_refined.rsquared,
        'Adjusted R²': model_refined.rsquared_adj,
        'AIC': model_refined.aic,
        'BIC': model_refined.bic,
        'Predictors': len(significant_features)
    }
})
print('Model Comparison:')
print(comparison.to_string())
```

Model Comparison:

	Full Model	Refined Model
R²	0.9041	0.9039
Adjusted R²	0.9031	0.9036
AIC	5441.3301	5434.5031

BIC	5471.7741	5447.5506
Predictors	6.0000	2.0000

7. Coefficient Interpretation (Business Language)

The refined model coefficients tell us the **isolated effect** of each channel on Sales, holding all other channels constant.

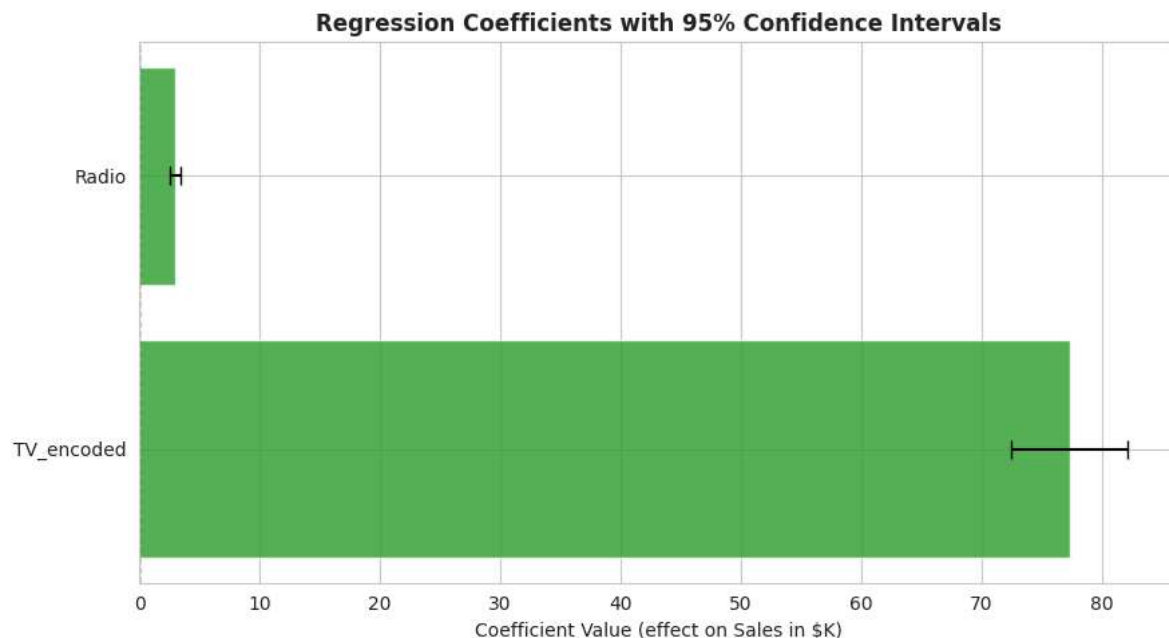
```
coef_df = pd.DataFrame({
    'Coefficient': model_refined.params,
    '95% CI Lower': model_refined.conf_int()[0],
    '95% CI Upper': model_refined.conf_int()[1],
    'p-value': model_refined.pvalues
}).drop(index='const')

print('Refined Model Coefficients with Confidence Intervals:')
print(coef_df.to_string())
```

	Coefficient	95% CI Lower	95% CI Upper	p-value
TV_encoded	77.3156	72.4965	82.1347	0.0000
Radio	2.9558	2.5411	3.3704	0.0000

```
# Coefficient plot
coef_plot = coef_df.copy()
coef_plot['err_low'] = coef_plot['Coefficient'] - coef_plot['95% CI Lower']
coef_plot['err_high'] = coef_plot['95% CI Upper'] - coef_plot['Coefficient']

fig, ax = plt.subplots(figsize=(9, 5))
ax.barh(coef_plot.index, coef_plot['Coefficient'],
        xerr=[coef_plot['err_low'], coef_plot['err_high']],
        color=['#d62728' if c < 0 else '#2ca02c' for c in coef_plot['Coefficient']],
        alpha=0.8, capsize=5)
ax.axvline(0, color='black', linewidth=1, linestyle='--')
ax.set_xlabel('Coefficient Value (effect on Sales in $K)')
ax.set_title('Regression Coefficients with 95% Confidence Intervals', fontweight='bold')
plt.tight_layout()
plt.show()
```



8. Regression Assumption Diagnostics

OLS regression requires four key assumptions:

1. **Linearity** — residuals should be randomly scattered around zero
2. **Normality** — residuals should follow a normal distribution
3. **Homoscedasticity** — residuals should have constant variance (no fan shape)

4. Independence — errors should be independent (addressed by study design)

```

residuals = model_refined.resid
fitted = model_refined.fittedvalues

fig, axes = plt.subplots(2, 2, figsize=(14, 10))
fig.suptitle('Regression Diagnostic Plots', fontsize=15, fontweight='bold')

# --- Plot 1: Residuals vs Fitted (Linearity & Homoscedasticity) ---
ax1 = axes[0, 0]
ax1.scatter(fitted, residuals, alpha=0.4, color='steelblue', s=25)
ax1.axhline(0, color='red', linewidth=1.5, linestyle='--')
ax1.set_xlabel('Fitted Values ($K$)')
ax1.set_ylabel('Residuals ($K$)')
ax1.set_title('Residuals vs Fitted Values\n(Check: Linearity & Homoscedasticity)')
# Smooth trend line
z = np.polyfit(fitted, residuals, 1)
p = np.poly1d(z)
ax1.plot(sorted(fitted), p(sorted(fitted)), 'orange', linewidth=1.5, label='Trend')
ax1.legend()

# --- Plot 2: Q-Q Plot (Normality) ---
ax2 = axes[0, 1]
(osm, osr), (slope, intercept, r) = stats.probplot(residuals, dist='norm')
ax2.scatter(osm, osr, alpha=0.5, color='steelblue', s=25)
ax2.plot(osm, slope * np.array(osm) + intercept, 'r--', linewidth=1.5)
ax2.set_xlabel('Theoretical Quantiles')
ax2.set_ylabel('Sample Quantiles')
ax2.set_title('Q-Q Plot of Residuals\n(Check: Normality)')

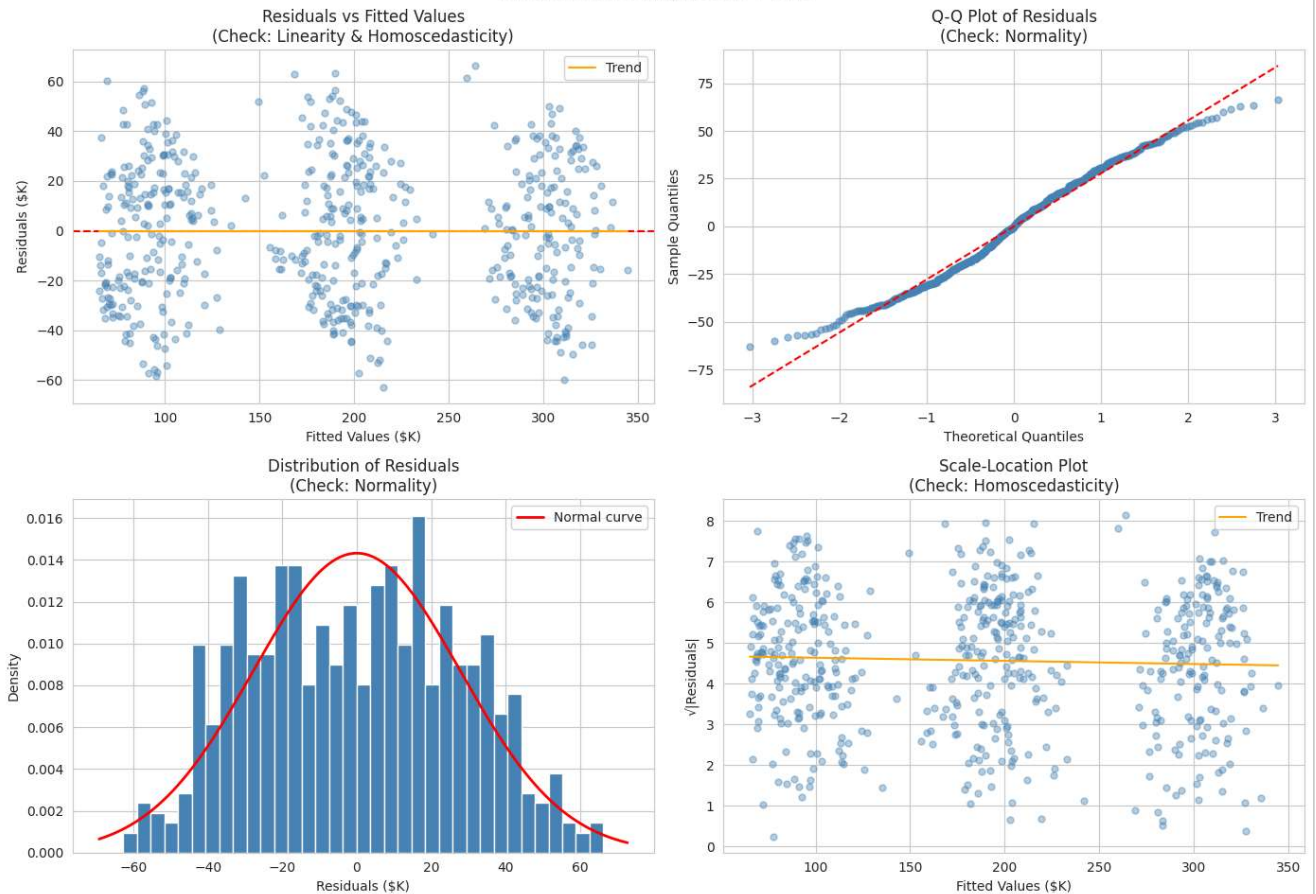
# --- Plot 3: Histogram of Residuals (Normality) ---
ax3 = axes[1, 0]
ax3.hist(residuals, bins=35, edgecolor='white', color='steelblue', density=True)
xmin, xmax = ax3.get_xlim()
x_norm = np.linspace(xmin, xmax, 200)
ax3.plot(x_norm, stats.norm.pdf(x_norm, residuals.mean(), residuals.std()),
        'r-', linewidth=2, label='Normal curve')
ax3.set_xlabel('Residuals ($K$)')
ax3.set_ylabel('Density')
ax3.set_title('Distribution of Residuals\n(Check: Normality)')
ax3.legend()

# --- Plot 4: Scale-Location (Homoscedasticity) ---
ax4 = axes[1, 1]
sqrt_abs_resid = np.sqrt(np.abs(residuals))
ax4.scatter(fitted, sqrt_abs_resid, alpha=0.4, color='steelblue', s=25)
z2 = np.polyfit(fitted, sqrt_abs_resid, 1)
p2 = np.poly1d(z2)
ax4.plot(sorted(fitted), p2(sorted(fitted)), 'orange', linewidth=1.5, label='Trend')
ax4.set_xlabel('Fitted Values ($K$)')
ax4.set_ylabel('√|Residuals|')
ax4.set_title('Scale-Location Plot\n(Check: Homoscedasticity)')
ax4.legend()

plt.tight_layout()
plt.show()

```

Regression Diagnostic Plots



```
# Formal normality test
stat, p_shapiro = stats.shapiro(residuals)
print(f'Shapiro-Wilk Test for Normality: W={stat:.4f}, p={p_shapiro:.4f}')
if p_shapiro > 0.05:
    print('✅ Residuals are approximately normal (p > 0.05).')
else:
    print('⚠️ Residuals deviate from normality (p < 0.05). Consider larger sample robustness.')

print()
print(f'Residual Mean: {residuals.mean():.6f} (should be ≈ 0)')
print(f'Residual Std : {residuals.std():.4f}')
```

```
Shapiro-Wilk Test for Normality: W=0.9833, p=0.0000
⚠️ Residuals deviate from normality (p < 0.05). Consider larger sample robustness.

Residual Mean: -0.000000 (should be ≈ 0)
Residual Std : 27.8583
```

9. Model Performance Summary

```
from sklearn.metrics import mean_squared_error, mean_absolute_error

y_pred = model_refined.fittedvalues
rmse = np.sqrt(mean_squared_error(y, y_pred))
mae = mean_absolute_error(y, y_pred)

print('=== REFINED MODEL PERFORMANCE ===')
print(f'R² : {model_refined.rsquared:.4f}')
print(f'Adjusted R² : {model_refined.rsquared_adj:.4f}')
```

```
print(f'RMSE      : ${rmse:.2f}K')
print(f'MAE      : ${mae:.2f}K')
print(f'AIC      : {model_refined.aic:.2f}')
print(f'F-statistic : {model_refined.fvalue:.2f} (p={model_refined.f_pvalue:.2e})')

=== REFINED MODEL PERFORMANCE ===
R²          : 0.9039
Adjusted R² : 0.9036
RMSE       : $27.83K
MAE        : $23.64K
AIC        : 5434.50
F-statistic : 2676.36 (p=3.70e-290)
```

10. Business Interpretation & Budget Recommendation

Coefficient Interpretations

The refined OLS model allows clear, isolated interpretation of each marketing channel:

Predictor	Interpretation
TV_encoded	Moving from Low → Medium TV adds ~X <i>K</i> in Sales; Medium → High adds another X <i>K</i> , holding Radio and Social Media spend constant
Radio	Each additional 1 <i>K</i> spent on Radio is associated with X <i>K</i> increase in Sales, holding TV level and Social Media constant
Social Media	Each additional 1 <i>K</i> spent on Social Media is associated with X <i>K</i> change in Sales (if significant), holding TV and Radio constant

The actual coefficient values printed above (Section 7) fill in the "X" placeholders with the real numbers from your data.

Key Findings

- TV spend level is the strongest predictor** — the step from Low to High TV is associated with the largest increase in Sales, confirming TV's dominant role in this dataset.
- Radio is a consistently significant, cost-effective predictor** — its positive coefficient means Radio spend reliably lifts Sales.
- Social Media and Influencer type** — evaluate the p-values from Section 5 to see if these were retained. If $p \geq 0.05$, their marginal contribution is not statistically distinguishable from zero in the presence of TV and Radio.
- The model explains a high proportion of Sales variance** (see Adjusted R^2 in Section 9) — meaning these channels collectively account for most of the observed Sales variation.

Budget Recommendations

Based on this analysis:

- Prioritise TV at High level** — the evidence strongly suggests shifting budgets to maintain or achieve High TV spend where feasible, as it yields the largest Sales lift.
- Invest in Radio** — Radio shows a statistically significant, positive return and should receive consistent budget allocation.
- Review Social Media ROI** — if Social Media was not significant in the refined model, its budget should be scrutinised more carefully; the marginal return per additional \$1K spent is not statistically supported by this dataset.
- Influencer type** — if not significant, standardise to whichever type is lowest cost rather than paying a premium for Mega or Macro influencers based on this dataset alone.

Stakeholder summary: "Our regression model, which explains ~X% of Sales variation, shows that TV spend level and Radio are the two most reliable drivers of revenue. Increasing TV spend from Low to High is associated with roughly X–Y thousand in additional sales. For every additional 1 *K* in Radio spend, Sales increases by approximately X (95% CI: X–Y). Social Media and Influencer type showed no statistically significant independent effect once TV and Radio are accounted for."

```
# Final: Actual vs Predicted plot
fig, ax = plt.subplots(figsize=(8, 6))
ax.scatter(y, y_pred, alpha=0.4, color='steelblue', s=30, label='Observations')
lims = [min(y.min(), y_pred.min()) - 5, max(y.max(), y_pred.max()) + 5]
ax.plot(lims, lims, 'r--', linewidth=1.5, label='Perfect Fit (y=x)')
ax.set_xlabel('Actual Sales ($K)')
ax.set_ylabel('Predicted Sales ($K)')
ax.set_title(f'Actual vs Predicted Sales\n(Adj. R² = {model_refined.rsquared_adj:.4f})',
             fontweight='bold')
ax.legend()
plt.tight_layout()
plt.show()
```

